

SOFTWARE ENGINEERING PRINCIPLES AND PRACTICES

WEEK-VIII: DESIGN

8.1 Objectives

Software Design is the process to transform the user requirements into some suitable form, which helps the programmer in software coding and implementation. During the software design phase, the design document is produced, based on the customer requirements as documented in the SRS document. Objectives of Software Design:



Correctness: A good design should be correct i.e. it should correctly implement all the functionalities of the system.

Efficiency: A good software design should address the resources, time, and cost optimization issues.

Understandability: A good design should be easily understandable, for which it should be modular and all the modules are arranged in layers.

Completeness: The design should have all the components like data structures, modules, and external interfaces, etc.

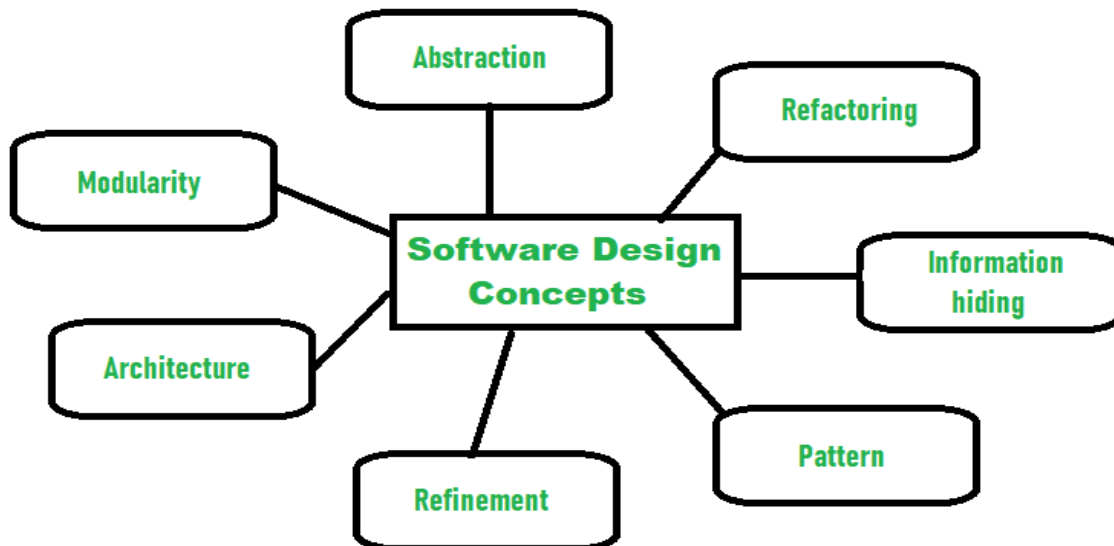
Maintainability: A good software design should be easily amenable to change whenever a change request is made from the customer side.

Consistency: There should not be any inconsistency in the design.

8.2 Design Concepts

The software design concept simply means the idea or principle behind the design. It describes how you plan to solve the problem of designing software, the logic, or thinking behind how you will design software. It allows the software engineer to create the model of the system or software or product that is to be developed or built. The software design concept provides a

supporting and essential structure or model for developing the right software. There are many concepts of software design and some of them are given below.



Abstraction simply means to hide the details to reduce complexity and increases efficiency or quality. The solution should be described in broad ways that cover a wide range of different things at a higher level of abstraction and a more detailed description of a solution of software should be given at the lower level of abstraction.

Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project. In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions.

Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.

Refinement simply means to refine something to remove any impurities if present and increase the quality. The refinement concept of software design is actually a process of developing or presenting the software or system in a detailed manner that means to elaborate a system or software. Refinement is very necessary to find out any error if present and then to reduce it.

Pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern. The pattern in the design process means the repetition of a solution to a common recurring problem within a certain context.

Information hiding simply means to hide the information so that it cannot be accessed by an unwanted party. In software design, information hiding is achieved by designing the modules in a manner that the information gathered or contained in one module is hidden and can't be accessed by any other modules.

Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features. Refactoring in software design means reconstructing the design to reduce complexity and simplify it without affecting the behavior or its functions.

8.3 Levels of design

Architectural Design: The architecture of a system can be viewed as the overall structure of the system & the way in which structure provides conceptual integrity of the system. The architectural design identifies the software as a system with many components interacting with each other. At this level, the designers get the idea of the proposed solution domain.

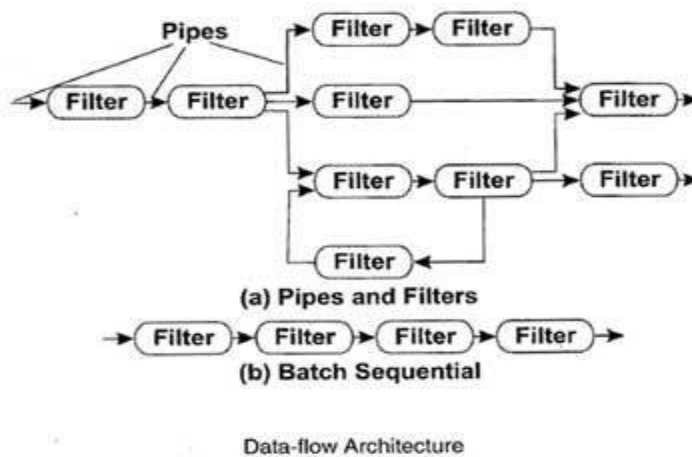
Preliminary or high-level design: Here the problem is decomposed into a set of modules, the control relationship among various modules identified, and also the interfaces among various modules are identified. The outcome of this stage is called the program architecture. Design representation techniques used in this stage are structure chart and UML.

Detailed design: Once the high-level design is complete, a detailed design is undertaken. In detailed design, each module is examined carefully to design the data structure and algorithms. The stage outcome is documented in the form of a module specification document.

8.4 Architectural styles

Architectural styles define a group of interlinked systems that share structural and semantic properties. In short, the objective of using architectural styles is to establish a structure for all the components present in a system.

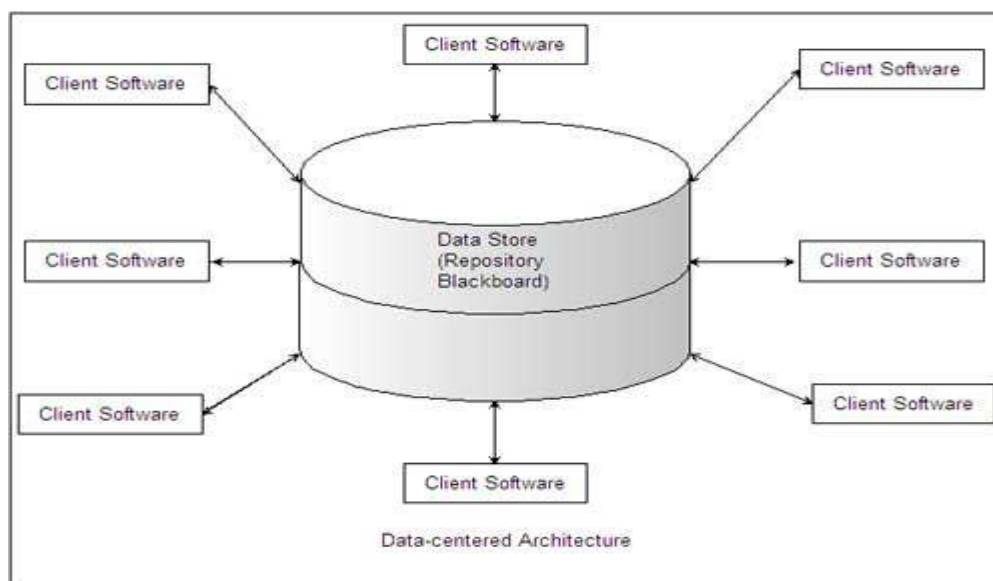
Data-flow architecture is mainly used in the systems that accept some inputs and transform it into the desired outputs by applying a series of transformations. Each component, known as filter, transforms the data and sends this transformed data to other filters for further processing using the connector, known as pipe.



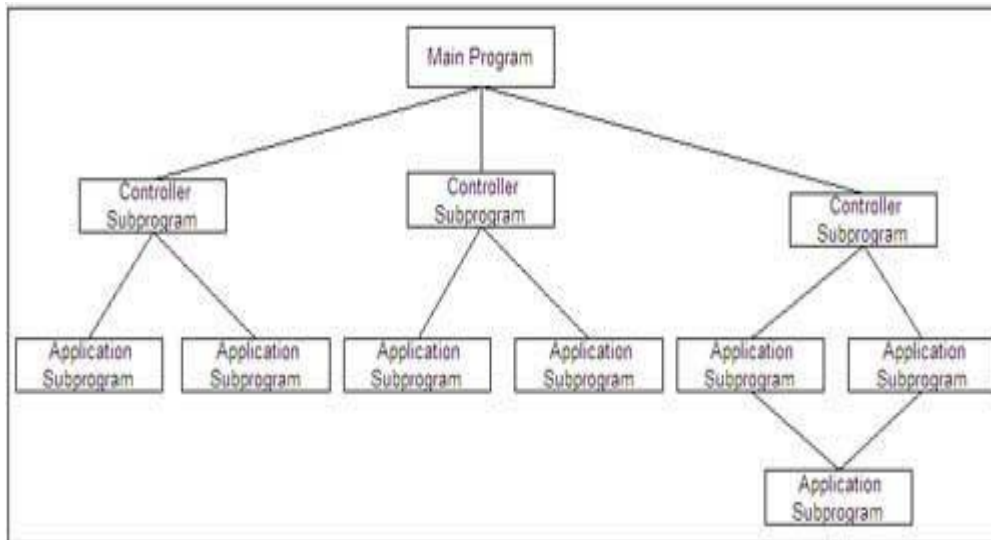
In **object-oriented architectural style**, components of a system encapsulate data and operations, which are applied to manipulate the data. In this style, components are represented as objects and they interact with each other through methods.

In **layered architecture**, several layers (components) are defined with each layer performing a well-defined set of operations. These layers are arranged in a hierarchical manner, each one built upon the one below it.

A **data-centered architecture** has two distinct components: a central data structure (data store) and a collection of client software. The data store represents the current state of the data and the client software performs several operations like add, delete, update, etc., on the data stored in the data store. In some cases, the data store allows the client software to access the data independent of any changes or the actions of other client software.



Call and return architecture enables software designers to achieve a program structure, which can be easily modified. This style consists of the following two sub styles.

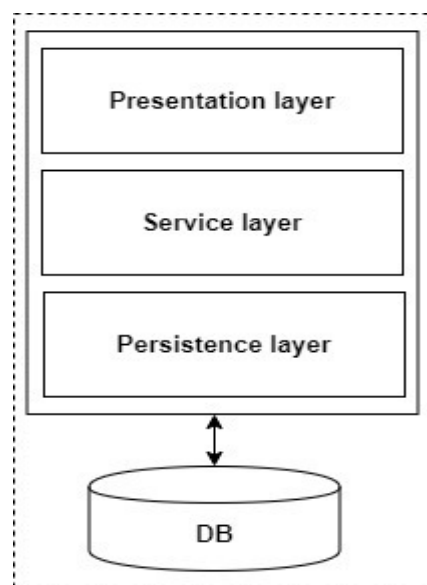


Main program/subprogram architecture: In this, function is decomposed into a control hierarchy where the main program invokes a number of program components, which in turn may invoke other components.

Remote procedure call architecture: In this, components of the main or subprogram architecture are distributed over a network across multiple computers.

8.5 Monolithic and Microservices

Monolithic: If all the functionalities of a project exist in a single codebase, then that application is known as a monolithic application. We design our application in various layers like presentation, service, and persistence and then deploy that codebase as a single jar/war file. This is nothing but a monolithic application, where “mono” represents the single codebase containing all the required functionalities.



Monolithic architecture

Advantages:

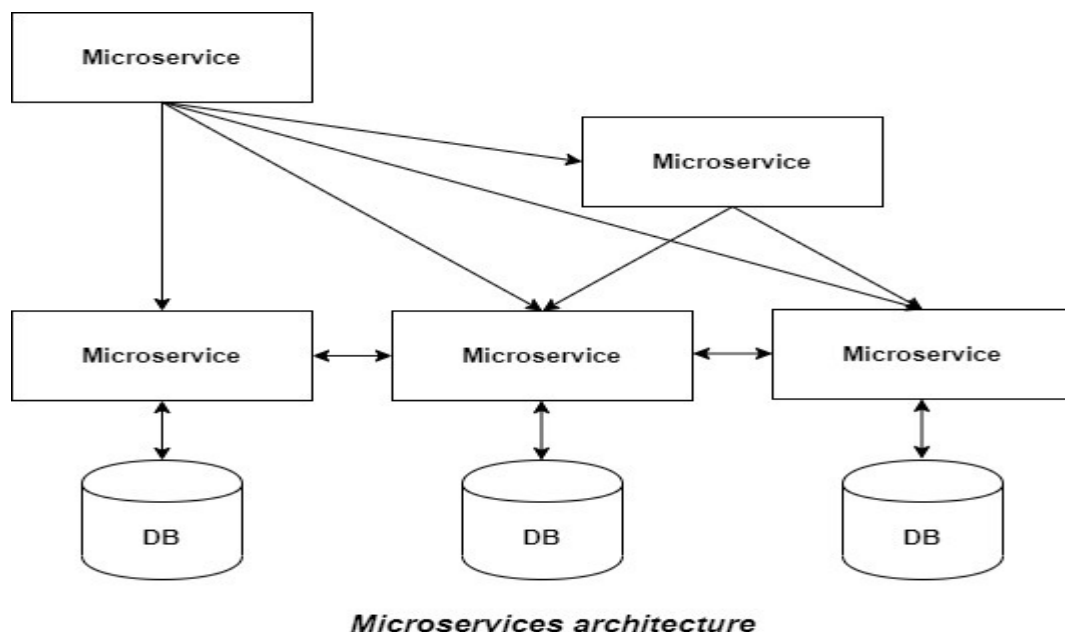
- Simple to develop
- Easier to deploy as only a single jar/war file is deployed
- The problems of network latency and security are relatively less architecture.
- Developers need not learn different applications, they can keep their focus on one application.

Disadvantages:

- It becomes too large with time and hence, difficult to manage.
- We need to redeploy the whole application, even for a small change.
- As the size of the application increases, its start-up and deployment time also increases.
- For any new developer joining the project, it is very difficult to understand the logic of a large application.
- It is very inefficient and takes up more resources unnecessarily.
- It is very difficult to adopt any new technology which is well suited for a particular functionality as it affects the entire application, both in terms of time and cost.

Microservices

It is an architectural development style in which the application is made up of smaller services that handle a small portion of the functionality and data by communicating with each other directly using lightweight protocols like HTTP.



The Microservice architecture has a significant impact on the relationship between the application and the database. Instead of sharing a single database with other microservices, each microservice has its own database.

Advantages:

- It is easy to manage as it is relatively smaller.
- If there's any update in one of the microservices, then we need to redeploy only that microservice.
- Microservices are self-contained and, hence, deployed independently.
- Their start-up and deployment times are relatively less.
- It is very easy for a new developer to onboard the project as he needs to understand only a particular microservice
- Each microservice can use different technology based on the business requirements.

Disadvantages:

- Being a distributed system, it is much more complex than monolithic applications.
- Its complexity increases with the increase in a number of microservices.
- Skilled developers are required to work with microservices architecture, which can identify the microservices and manage their inter-communications.
- Independent deployment of microservices is complicated.
- Microservices are costly in terms of network usage as they need to interact with each other and all these remote calls result in network latency.

8.6 Overview UI

User interface (UI) design refers to the visual design of a product's digital interface, such as an app screen or website. It's the process of designing the visual and interactive properties of a visual experience, ensuring the interface is both clear for the user to navigate and overall aesthetically pleasing. UI design is a multidisciplinary field involving various elements from visual design, UX design, and graphic design.

UI design involves the complex art of arranging these elements to guide the user and keep cognitive load to a minimum while creating beautiful, unique experiences that bring the product to life.

8.7 Overview UX

UX design, or user experience design, is the process of creating products and experiences that help users get done what they've come to do—as efficiently and effectively as possible. The user experience is how users feel as they engage with your product. This concept applies to physical objects and daily experiences, like using a coffee maker or buying a car at a dealership; but it also applies to the digital side of life—think about the last time you transferred money using a banking app, purchased something online, or even played a game on your phone.

- **What is UX (User Experience)?**

The user experience (UX) is what a user of a particular product experiences when using that product. The process includes development, improvement of quality interaction between a user and all facets of a company. The full form of UX design is User Experience Design. UX design improves the overall experience a user has with a company's products or services.

- **What is UI (User Interface)?**

UI design relates to the appearance and feel of a digital product. It focuses on visual factors like buttons, fonts, colour schemes, images, interactive elements, etc. The full form of UI design is the User Interface Design. It is developing and improving the quality of interaction between user and software.

The difference between User Interface (UI) and User Experience (UX) is that UI refers to the aesthetic elements by which people interact with a product, while UX is about the experience a user has with a product or service.

UX Design

- UX Design stands for User Experience Design.
- UX design is focused on everything that affects the user's journey to solve a problem.
- UX design deals with research, testing, development, content, and prototyping.
- UX design is developing and improving quality interaction between a user and all elements of a company.
- UX design is a complete experience which may not be limited to the screen.
- It involves creative and convergent thinking.
- UX design is based on the client's needs and requirements.

UI Design

- UI Design stands for User Interface Design.
- UI design is a process that mainly focused on how the specific product's surfaces look and function.
- UI is a process of visually guiding the user through a product's interface using interactive elements across all platforms.
- UI design transmits the brand's strength and visual assets to a product's interface.

- UI design is usually visual design and information design around screens.
- It involves creative and critical thinking.
- UI design is based on the user's needs and research.
- UI design needs wireframes, prototyping, and a good research approach.

Choosing Interface Elements

Users have become familiar with [interface elements](#) acting in a certain way, so try to be consistent and predictable in your choices and their layout. Doing so will help with task completion, efficiency, and satisfaction.

Interface elements include but are not limited to:

- **Input Controls:** buttons, text fields, checkboxes, radio buttons, dropdown lists, list boxes, toggles, date field
- **Navigational Components:** breadcrumb, slider, search field, pagination, tags, icons.
- **Informational Components:** tooltips, icons, progress bar, notifications, message boxes, modal windows.
- **Containers:** accordion

There are times when multiple elements might be appropriate for displaying content. When this happens, it's important to consider the trade-offs. For example, sometimes elements that can help save you space, put more of a burden on the user mentally by forcing them to guess what is within the dropdown or what the element might be.

UI types

There are many different types of interface that might come with an operating system. These are:

1. Graphical User Interfaces (GUI)
2. Command Line Interfaces (CLI)
3. Form-based interfaces
4. Menu-based interfaces
5. Touch user interface
6. Voice user interface

Graphical User Interfaces (GUI)

Interfaces that are graphical in nature are known either as Graphical User Interfaces (**GUI**) or **WIMP** interfaces (Windows, Icons, Menus and Pointer). Typically, you would expect these types of interfaces to be available in multi-tasking environments (where you open and use more than one piece of software at a time) or in applications software that involve a considerable degree of complexity. You will all have used a GUI hundreds of times, when you used Windows.

Command Line Interface

A command line interface requires a user to type in commands from a list of allowable commands.

Form-based interfaces

Form-based interfaces are very suitable for any application that involves entering predictable pieces of information into the computer.

Menus

Some operating systems are designed with a menu-based user interface. Menu-based user interfaces are ideal for situations where the user's IT skills cannot be guaranteed or in situations which require selections to be made from a very wide range of options or in situations which require very fast selection.

Touch user interface:

User interface through haptics or touch. Most smartphones, tablets and any device that operates using a touch screen use haptic input.

Voice user interface:

Interactions between humans and machines using auditory commands. Examples include virtual assistant devices, talk-to-text and GPS.

Important qualities of User Interface Design are following:

1. Simplicity :

- User Interface design should be simple.
- Less number of mouse clicks and keystrokes are required to accomplish this task.
- It is important that new features only add if there is a compelling need for them and they add significant values to the application.

2. Consistency :

- The user interface should have a more consistency.
- Consistency also prevents online designers' information chaos, ambiguity and instability.
- We should apply typeface, style and size convention in a consistent manner to all screen components that will add screen learning and improve screen readability. In this we can provide permanent objects as unchanging reference points around which the user can navigate.

3. Intuitiveness :

- The most important quality of good user interface design is intuitive.
- Intuitive user interface design is one that is easy to learn so that user can pick it up quickly and easily.
- Icons and labels should be concise and cogent. A clear unambiguous icon can help to make user interface intuitive and a good practice is making labels conform to the terminology that the application supports.

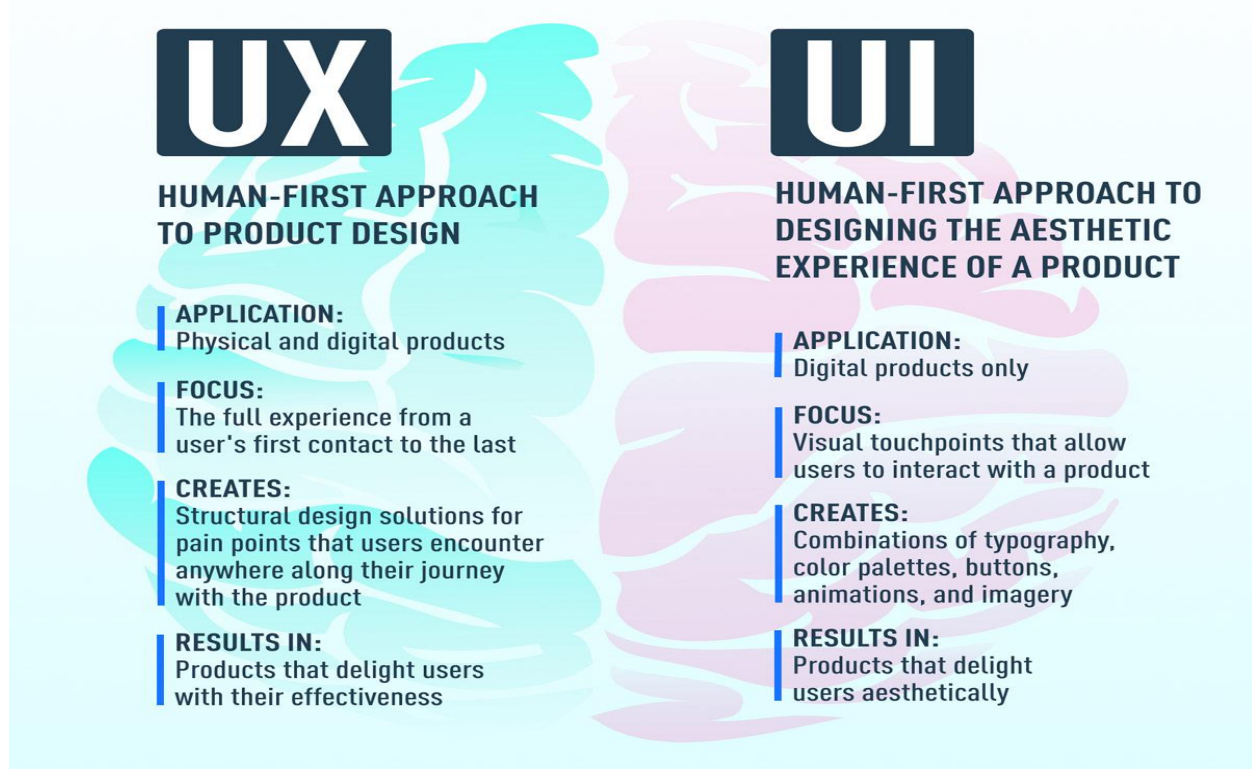
4. Prevention:

- A good user interface design should prevent users from performing an inappropriate task and this is accomplished by disabling or "graying out" certain elements under certain conditions.

5. Forgiveness:

- This quality can encourage users to use the software in a full extent.
- Designers should provide users with a way out when users find themselves somewhere they should not go.

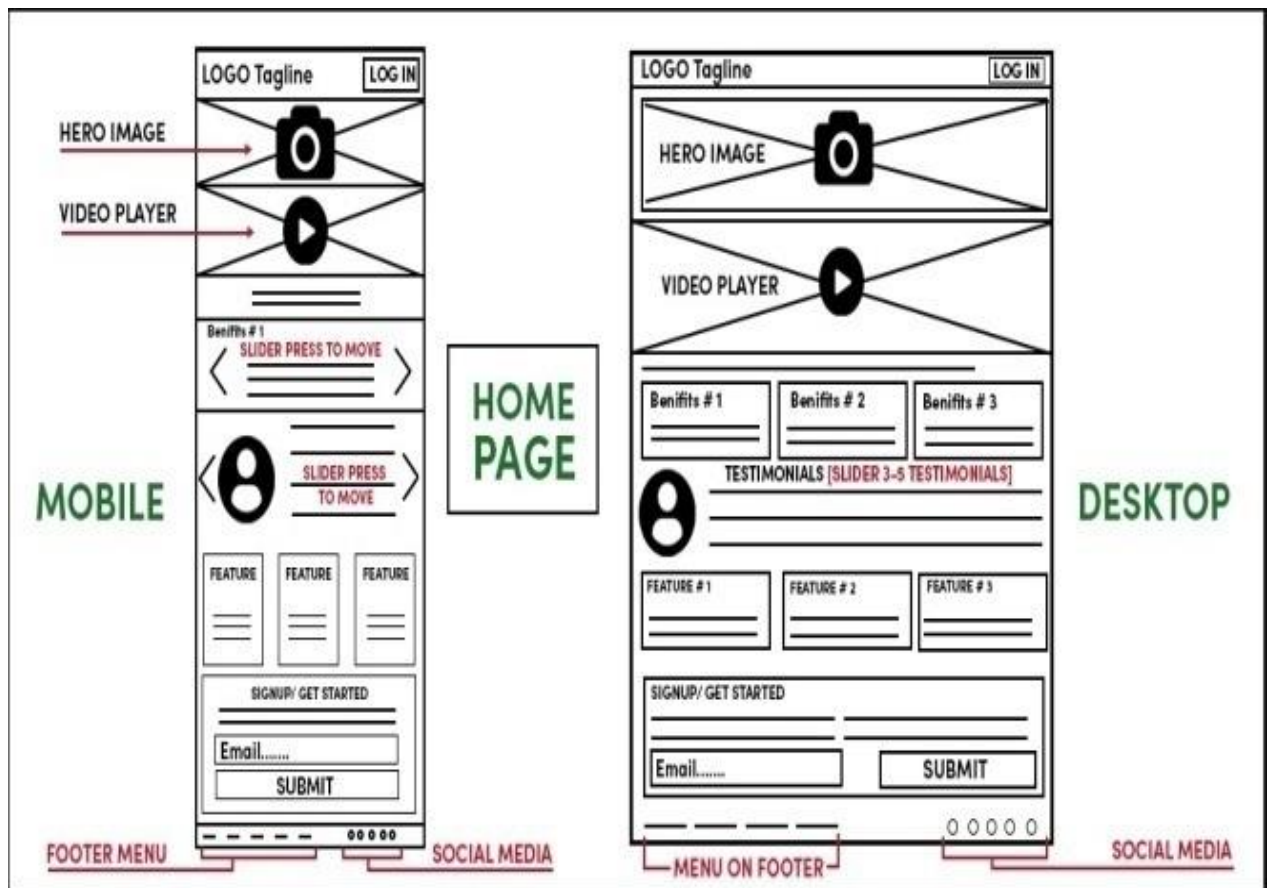
The Relationship between UI and UX



Wireframes:

Overview

- A wireframe is a schematic or blueprint that is useful for helping you, your programmers and designers think and communicate about the structure of the software or website you're building.



- A wireframe is commonly used to layout content and functionality on a page which takes into account user needs and user journeys.
 - Wireframes are used early in the development process to establish the basic structure of a page before visual design and content is added.
 - Let's say that you've researched your users, and established a user flow. You also now have a good understanding of what elements have to be on each screen. What's next? It's time to start creating.
 - Think of a wireframe as a low-fidelity blueprint for your product. It's meant to be a step-by-step guide of your customer's journey. These blueprints can be executed both digitally and in pencil.
 - While drawing your wireframes is a fun exercise, it's becoming more common to create them digitally in **Sketch, AdobeXD, and Figma.**

Purpose:

Wireframes serve three key purposes: They keep the concept **user-focused**, they **clarify and define website features**, and they are **quick and cheap to create**.

1. Wireframes keep the concept user-focused

- Wireframes are effectively used as communication devices
- They facilitate feedback from the users, instigate conversations with the stakeholders, and generate ideas between the designers.
- Conducting user testing during the early wireframing stage allows the designer to honest feedback, and identify key points that help to establish and develop the product concept.
- Wireframing is the perfect way for the designers to gauge how the user would interact with the interface.
- These insights help the designer to understand what feels intuitive for the user, and create products that are comfortable and easy to use

2. Wireframes clarify and define website features

- Wireframing specific features will clearly communicate to your clients how they'll function and what purpose they'll serve.
- It also enables all stakeholders to gauge how much space will need to be allocated for each feature and clarify the page's functionality , architecture of visual design
- Seeing the features on a wireframe will also allow you to visualize how they all work together—and may even prompt you to decide to remove a few if you feel they're not quite working with the rest of the page's elements.

3. Wireframes are quick and cheap to create

- In fact, if you have a pen and paper to hand, you can quickly sketch out a wireframe without spending a penny.
- The abundance of tools available means you can also build a digital wireframe

within minutes

- Often, when a product seems too polished, the user is less likely to be honest about their first impressions.
- But by exposing the very core of the page layout, flaws and pain points can easily be identified and rectified without any significant expenditure of time or money.
- The later it gets in the product design process, the harder it is to make changes!

8.3.4.3 Benefits:

1. Make Changes More Efficiently
2. Display the Architecture of Your Site
3. Clarify Website Features before You Build Them
4. Focus on the User Experience
5. Determine Website Responsiveness
6. Make Content Development Better
7. Save Time and Money